

AWS in Plain English · [Follow publication](#)

★ Member-only story

Stop Managing Docker Like It's 2020: Three Tools That Changed Everything

Smarter, Faster, and Stress-Free Docker Management



Dipanshu · [Follow](#)

Published in AWS in Plain English

5 min read · Feb 27, 2025



Listen



Share

... More

Non-Medium Members Can Read the Full Article by Clicking Here.

Docker simplifies containerization, but let's be honest — managing containers can feel like wrestling with an octopus. Constantly running `docker ps`, manually cleaning up images, or debugging why a container is behaving strangely can be frustrating.

Want to save time, reduce stress, and boost productivity? Meet three amazing tools: **LazyDocker**, **Dive**, and **WatchTower** game changing tools that will save you time, reduce headaches, and boost productivity. Let's dive in and see how they can transform your Docker experience!

LazyDocker : Stress-Free Container Management

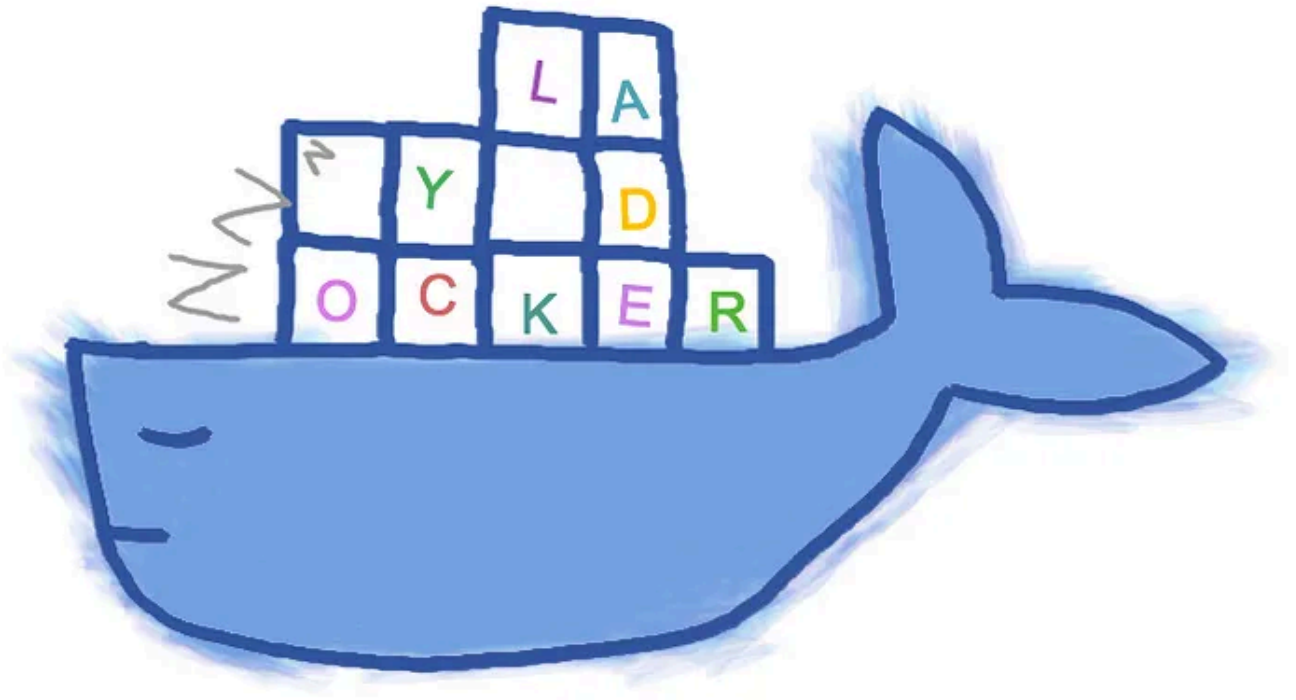


Image from [lazydocker](#)

I started using Lazydocker for container management a while ago and I'm not going back.

- All services in one simple view.
- Logs are clean and easy to follow.
- Restart or rebuild with just a keypress.
- Managing Docker finally feels stress free.

No more `docker-compose up` locking my terminal or logs stopping with crashes!

What is Lazy Docker ?

Lazy Docker is a terminal UI for Docker and Docker Compose that makes managing containers a breeze. Instead of having to remember and type out long Docker commands, you get an interactive interface where everything is just a keypress away.

Some of the key features include:

- View container status, logs, and metrics at a glance
- Restart/remove/rebuild containers with a single keypress
- Monitor resource usage with ASCII graphs

- Attach to container shells easily
- Prune unused containers, images and volumes
- Full mouse support for clicking and scrolling

Getting Started

Here's how you can install and run LazyDocker:

On Mac

```
brew install lazydocker  
lazydocker
```

then you can just run `lazydocker` container to start it.

```
docker run --rm -it -v \  
/var/run/docker.sock:/var/run/docker.sock \  
-v /yourpath:/.config/jesseduffield/lazydocker \  
lazyteam/lazydocker
```

(Don't forget to replace `/yourpath` with the path to your home directory.)

Dive : Analyze Your Docker Images

Have you ever wondered what's actually inside your Docker images? Or why they're so large?

Dive is an essential tool for analyzing Docker images. It allows you to inspect the contents of your images layer by layer, helping you understand what's inside your containers. This insight can lead to better optimization strategies for your applications.

What is Dive?

Dive is a tool used for exploring Docker images, layer contents, and discovering ways to shrink your Docker image size. It presents a layer-by-layer view of images, making it easy to understand how each instruction in your Dockerfile affects the final image.

Some of the key features include:

- Interactive layer exploration with detailed size information
- Efficiency score calculation for your images (the higher the better)
- Identification of duplicated files across layers
- Analysis of wasted space in your images (the lower the better & “this is what we want to reduce”)
- Command history for each layer
- Integration with CI/CD pipelines

How to use Dive?

To analyze a Docker image simply run dive with an image tag/id/digest:

```
dive <your-image-tag>
```

Pro Tip: No Installation Required

Don't want to install another tool? No problem! Here's a handy alias that runs Dive in a container:

```
alias dive="docker run -ti --rm -v /var/run/docker.sock:/var/run/docker.sock w  
dive <your-image-tag>
```

WatchTower: Automatic Updates for Peace of Mind

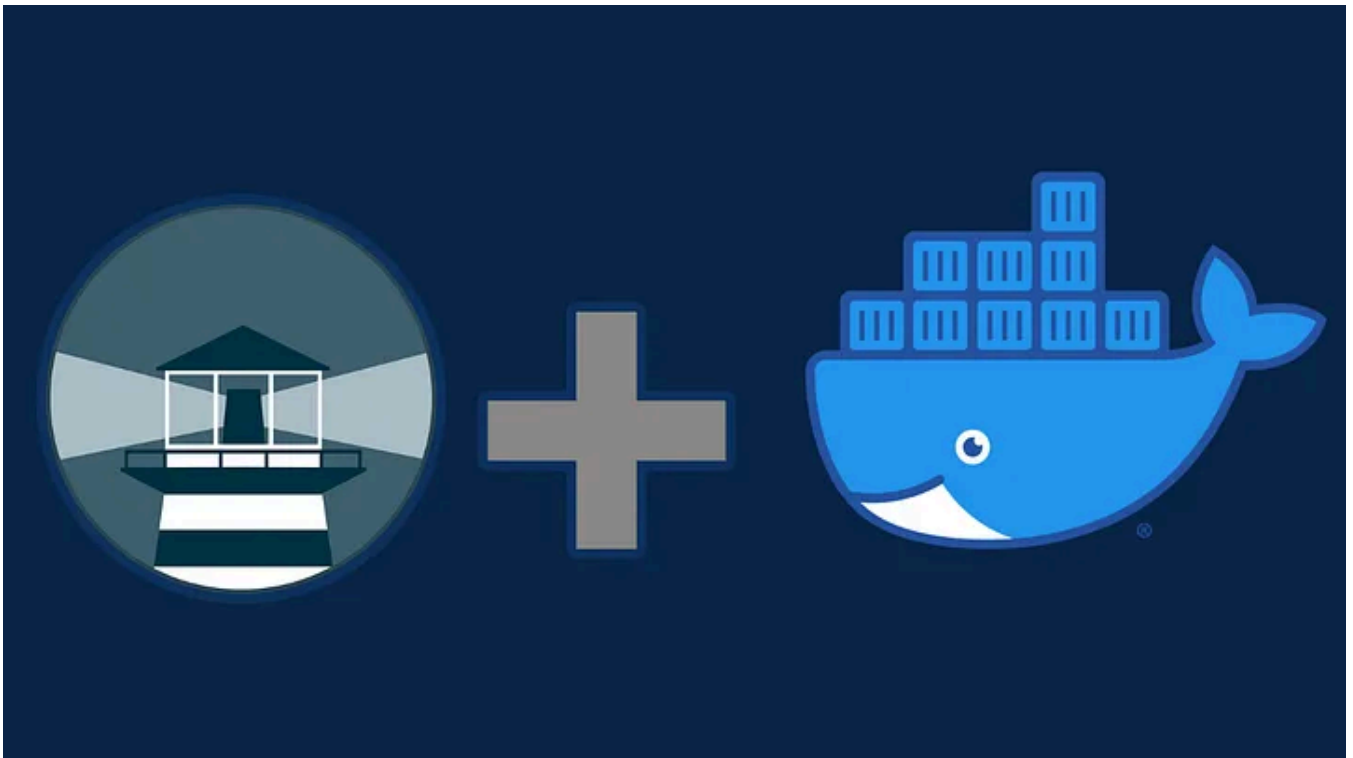


Image by author

What is WatchTower?

WatchTower monitors your running Docker containers and automatically updates them when their base image changes. This automation ensures your containers always run the latest versions, improving security and reliability.

Some of the key features include:

- **Monitoring:** Constantly watches containers for image version changes.
- **Detecting Updates:** Prompts alerts on new container image versions to prevent outdated releases.
- **Updating Containers:** Swiftly fetches and updates containers with fresh images, ensuring code stays current.

Embrace Watchtower for reduced overhead, increased security, and improved stability:

- **Reduced Overhead:** Eliminate tedious manual updates and focus on building instead.
- **Increased Security:** Patches vulnerabilities by ensuring containers run on the latest versions.

- **Improved Stability:** Ensures applications benefit from performance optimizations and bug fixes.

You can try it out with the following command:

```
docker run -d \  
  --name watchtower \  
  -e REPO_USER=username \  
  -e REPO_PASS=password \  
  -v /var/run/docker.sock:/var/run/docker.sock \  
  containrrr/watchtower container_to_watch --debug
```

When using these tools together, consider the following best practices:

1. Use LazyDocker for Development

- Keep it open during development for quick access to logs and container management
- Create custom keybindings for frequent operations

1. Optimize with Dive

- Run Dive analysis before pushing images to production
- Set efficiency score thresholds in CI/CD pipelines
- Use multi-stage builds based on Dive's feedback

1. Configure WatchTower Wisely

- Start with monitoring only specific containers
- Implement proper notification systems
- Schedule updates during low-traffic periods

Why These Tools Matter ?

These tools are not just about saving time — they enhance your productivity, reduce errors, and help you focus on building and deploying applications rather than fighting with container management.

- **LazyDocker:** Simplifies day-to-day management.
- **Dive:** Optimizes images for faster builds and deployments.
- **Watchtower:** Keeps your containers updated automatically.

Start integrating these tools into your Docker workflow, and experience a seamless container management process.

Let's Connect!

Thanks for reading this article through the end!

If you enjoyed this article, tap that clap 🖐️ button 50 times! follow me on [X](#) for more insights and [Subscribe to my newsletter](#) for FREE to be notified of new articles! Your thoughts and feedback are always welcome in the comments below- I read them all!.

Get an email whenever Dipanshu publishes

Get an email whenever Dipanshu publishes. By signing up, you will create a Medium account if you don't already have...

medium.com

Further Reads

You might also find these articles interesting:

Docker pros are shrinking images by 99%: The hidden techniques you can't afford to miss

Unlock the secrets to lightning-fast deployments and slashed costs — before your competitors do

aws.plainenglish.io

Why Developers Are Ditching PostgreSQL, MySQL and MongoDB

The Database Revolution Your Project Is Missing Out On

aws.plainenglish.io

Thank you for being a part of the community

Before you go:

- Be sure to **clap** and **follow** the writer 🙌

Open in app ↗

Medium

🔍 Search



- [Start your own free AI-powered blog on Differ](#) 🚀
- [Join our content creators community on Discord](#) 👤 💻
- For more content, visit [plainenglish.io](#) + [stackademic.com](#)

Docker

AWS

DevOps

Programming

Containers

AWS

Follow

Published in AWS in Plain English

28K Followers · Last published 1 day ago

New AWS, Cloud, and DevOps content every day. Follow to join our 3.5M+ monthly readers.



Follow

Written by Dipanshu

821 Followers · 12 Following

AI researcher and enthusiast, always learning and intrigued by the world of artificial intelligence.

Responses (13)



Sandesh Chand

What are your thoughts?



Pacorro

Feb 28



Fantastic read !!!



14



1 reply

[Reply](#)



Fraser Goffin

Mar 16



Auto updating containers when a base image changes .. in a production (well actually any) environment ? ... I think not !

Here's a quote from the WatchTower docs ...

'Watchtower is **intended to be used in homelabs**, media centers, local dev environments... [more](#)



53

[Reply](#)



dibu

Feb 28



The easiest for me is Dockge. Is a simple and powerful web ui for Docker Compose.

It is as simple as copy/paste. And if you only have docker run command there is converter for it.



16



1 reply

[Reply](#)

[See all responses](#)

More from Dipanshu and AWS in Plain English

SaaS Killer



 Dipanshu

Paying for software is stupid ..Open-Source tools to Destroy Your SaaS Expenses

These 40 Open-Source Tools Will Make Your SaaS Subscriptions Look Obsolete

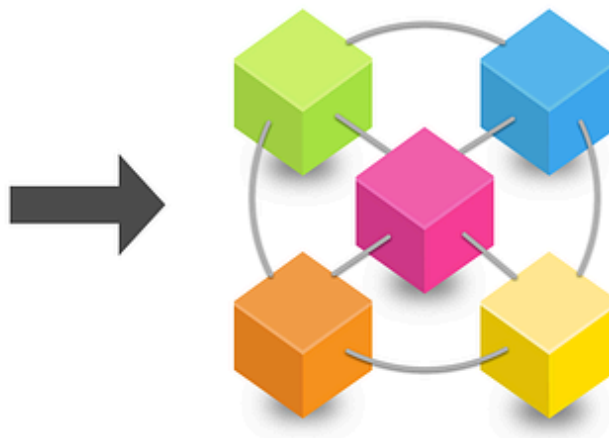
★ 6d ago 🖱️ 1K 💬 11



Monolithic



Microservices





In AWS in Plain English by Coders Stop

Why Your Microservices Architecture Is a Ticking Time Bomb

“Let’s break this monolith into microservices!”



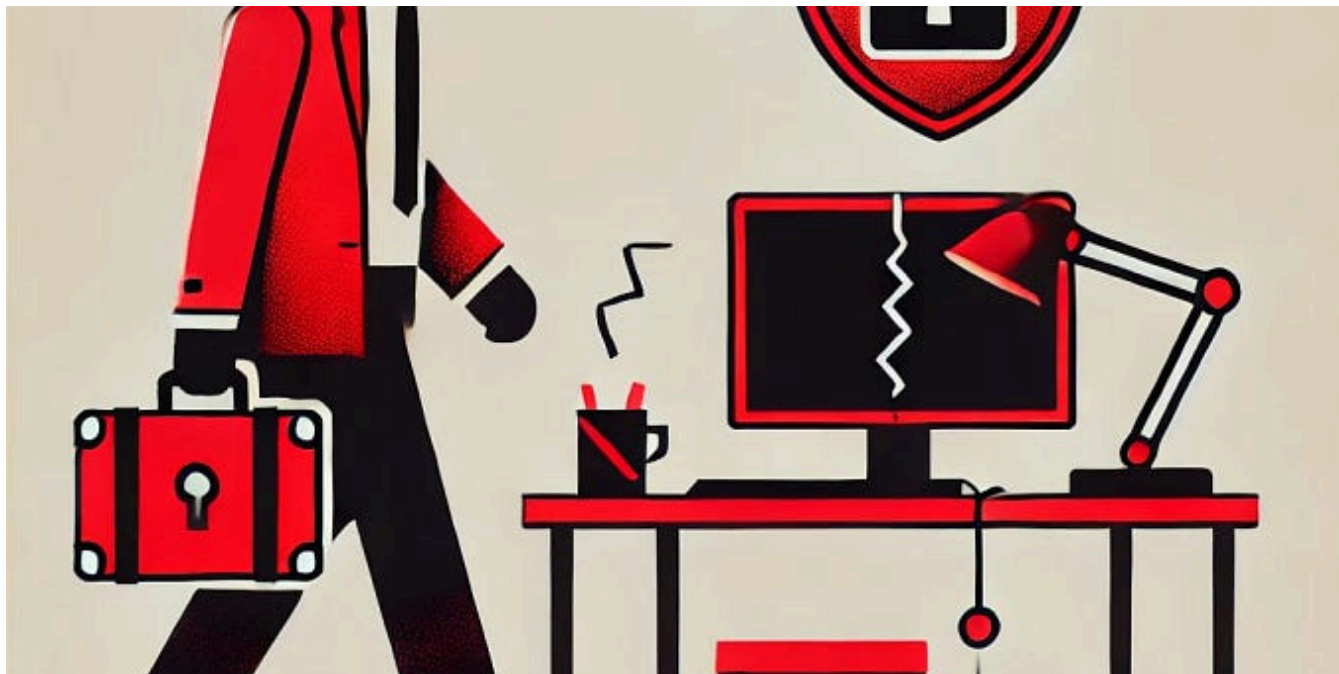
Mar 21



297



7



In AWS in Plain English by Taimur Ijlal



Why Are Cybersecurity Professionals Losing Their Jobs In 2025 ?

Layoffs Do Not Define You. How To Keep Moving Forward



Feb 18

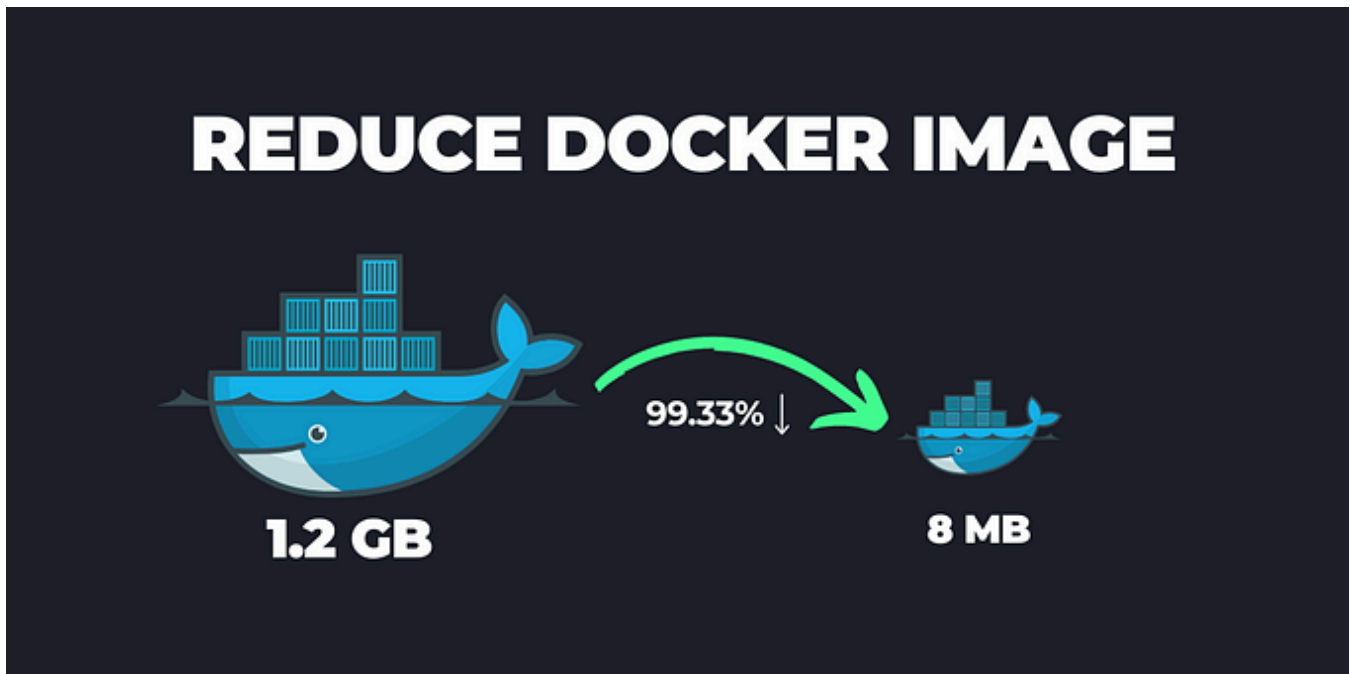


301



17





 In AWS in Plain English by Dipanshu

Docker pros are shrinking images by 99%: The hidden techniques you can't afford to miss

Unlock the secrets to lightning-fast deployments and slashed costs—before your competitors do

★ Sep 18, 2024 🖱️ 4.4K 💬 26



See all from Dipanshu

See all from AWS in Plain English

Recommended from Medium



Sebastian Carlos

Fired From Meta After 1 Week: Here's All The Dirt I Got

This is not just another story of a disgruntled ex-employee. I'm not shying away from the serious corporate espionage or the ethical...



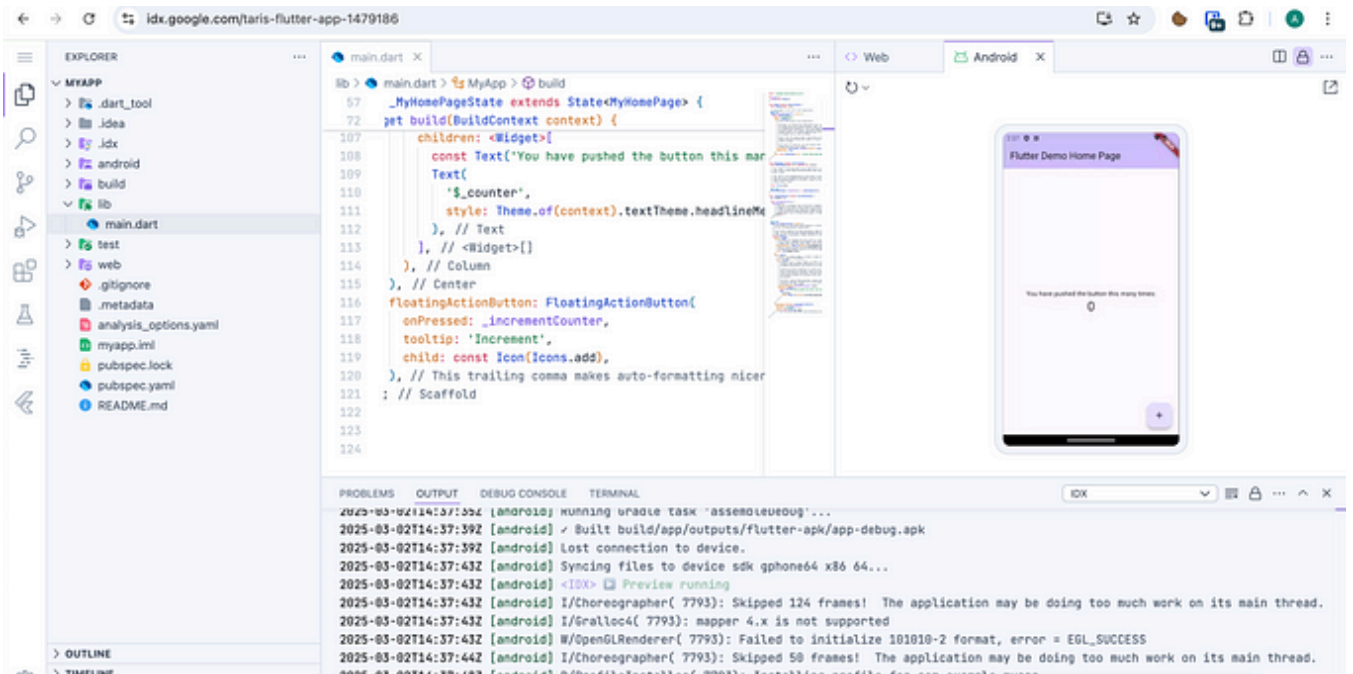
Jan 8



19K



417



In Coding Beauty by Tari Ibaba

This new IDE from Google is an absolute game changer

This new IDE from Google is seriously revolutionary.

★ Mar 11 🖱 3K 💬 183



11 OPEN-SOURCE PROJECTS THAT ARE LEGIT GAME-CHANGERS

 In Let's Code Future by Let's Code Future

11 Open-Source Projects That are Legit Game-Changers

Really unique and valuable

★ Mar 24 🖱 996 💬 14



```
26 func (l *Listener) listenToQueue(ctx context.Context, topic string, queue MessageQueue) {
51
52     // set the callback function for this event to commit the message to kafka
53     wrapper.SetCallback(func(c context.Context) {
54         commitToKafka(c, queue, msg)
55     })
56
57     l.workerPool.Submit(func() {
58         if err := l.eventHandler.HandleEvent(cctx, wrapper); err != nil {
59             log.Error(cctx, "Error handling event", log.Tags{"topic": topic,
60                 "error": err})
61             wrapper.DoCallback() // commit the message to kafka
62         }
63     })
64
65     // Wait for an OS signal to exit
66     sigchan := make(chan os.Signal, 1)
67     signal.Notify(sigchan, syscall.SIGINT, syscall.SIGTERM)
68     <-sigchan
69
70     log.Info(cctx, "Received signal, shutting down...", log.Tags{"topic": topic})

```

I added this code block but copied it blindly from the example given in the segmentio/kafka-go docs. Is this the correct way to handle this? Or is there a better way?

The signal handling code you've added is problematic in its current location. It's inside the `listenToQueue` method which runs as a goroutine for each queue, meaning you'll have multiple signal handlers competing with each other. Additionally, this signal handling should be at a higher level in your application.

Here's how to fix it:

```
listen.go
func (l *Listener) Run(ctx context.Context) error {
    for topic, queue := range l.queues {
        go l.listenToQueue(ctx, topic, queue)
    }
    <-ctx.Done()
    return nil
}

func (l *Listener) listenToQueue(ctx context.Context, topic string, queue
    defer queue.Close()

    for {

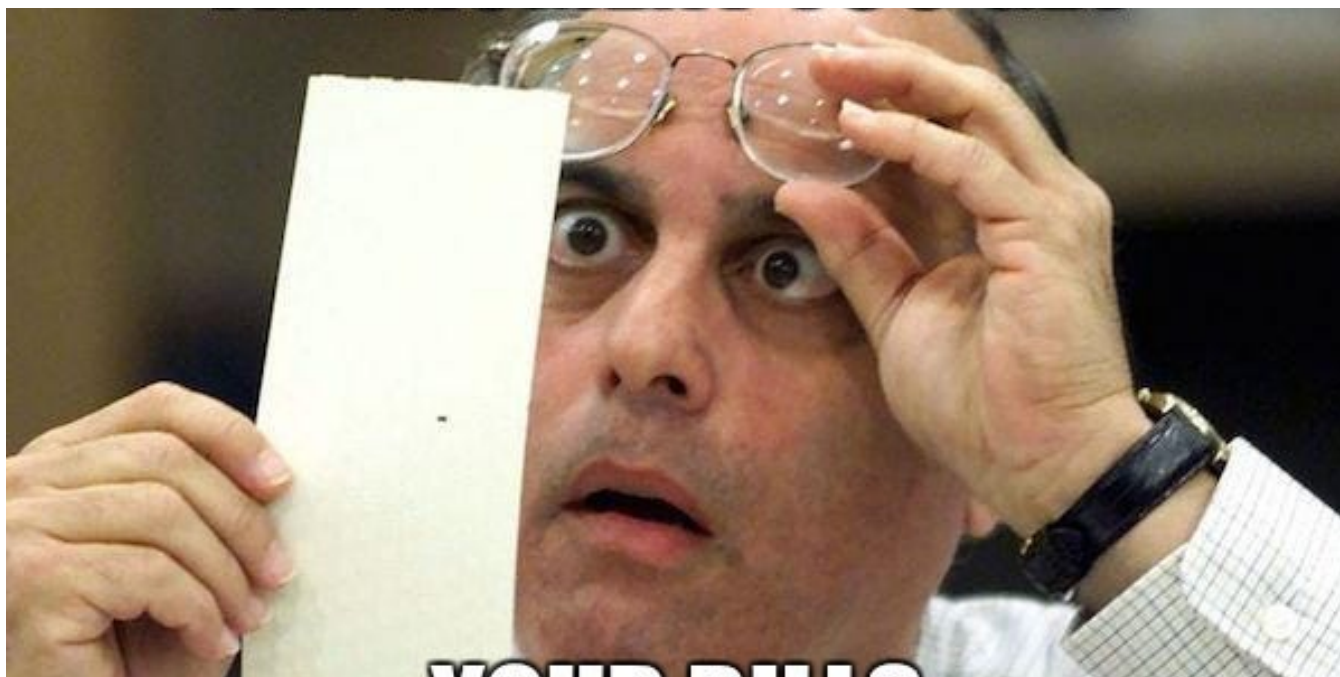
```

 In Level Up Coding by Jacob Bennett

The 5 paid subscriptions I actually use in 2025 as a Staff Software Engineer

Tools I use that are cheaper than Netflix

★ Jan 7 🖱 11.8K 💬 290

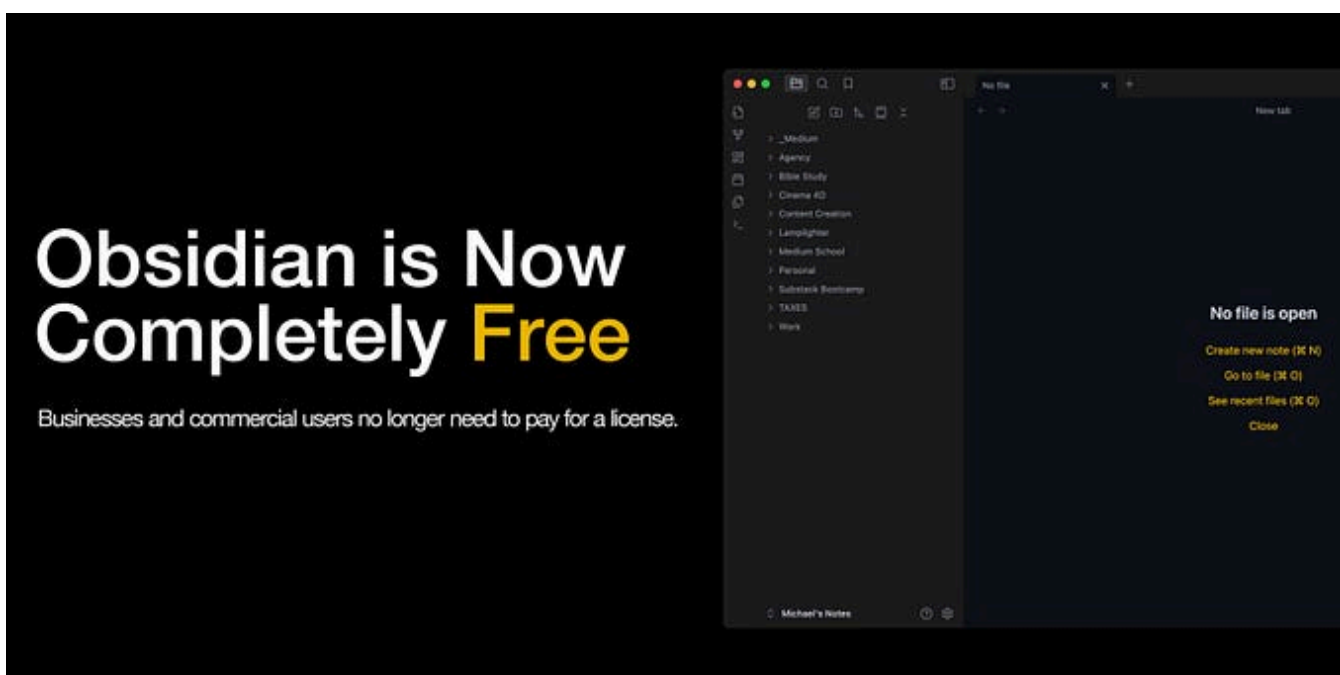


👤 In Data Engineer Things by Yingjun Wu

BigQuery's Ridiculous Pricing Model Cost Us \$10,000 in Just 22 Seconds!!!

Yes, you read that right. \$10,000 burned in less than half a minute.

Mar 19 🖱 917 💬 41





Michael Swengel

Obsidian is Now Completely FREE to Use—Even for Businesses

But there are other ways to support Obsidian if you want



Feb 25



239



6



See more recommendations